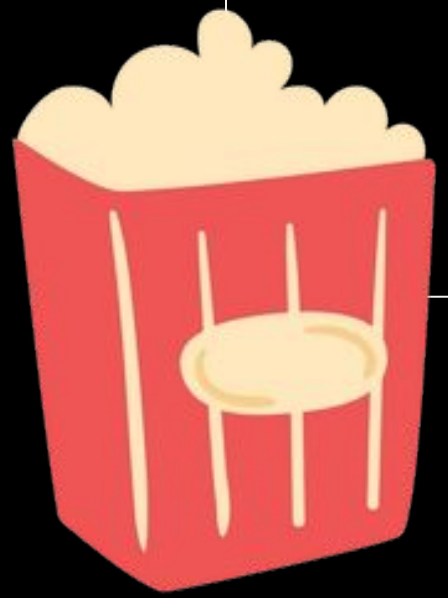
Popcoders Project Presentation



Members and Roles

Role:

- ➔ Reviews table and page
- ➔ Home Page

Mohammed AlAashi
150240935
alashim24@itu.edu.tr

Role:

- ➔ Search logs table and page
- ➔ User table

Eman El Falouji
150240936
alfaloujie4@itu.edu.tr

Members and Roles

Role:

- ➔ Watch history table and page

Jane Al-Shihabi
912500228
shihabi25@itu.edu.tr

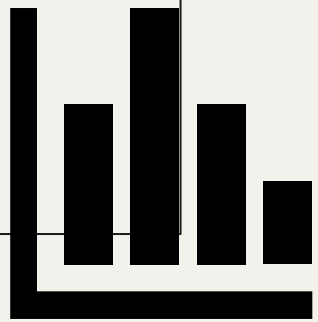
Role:

- ➔ Recommendation table and page
- ➔ Movies page

Mustafa Furkan Delioğlu
150220734
delioglu22@itu.edu.tr

Overview of Application

➔ **Popcoders** is a web-based movie tracking application that presents data on user viewing behavior, allows users to manage movie reviews, and provides data-driven insights such as recommendations through data analysis.



Overview of Application

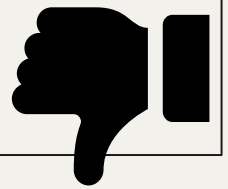
Recommendations engine	Watch history tracking	Movie reviews system
personalized movie suggestions based on viewing patterns	access data on viewing sessions with duration, progress, and location data	users can rate movies (0-5 stars) and write reviews
Search analytics	**User management**	**Analytics dashboard**
track search queries and view user behavior patterns	manage user accounts with profiles and activity status	aggregated data using complex SQL queries

\+ a page to view all movies with their data

Motivation & Problem Statement

- Streaming platforms need to understand viewer behavior
- Data-driven info to improve content recommendations
- Track what users watch, search, and rate
- Analyze viewing patterns across different regions

Problem Statement: In order to increase profitability of streaming platforms, especially with monopolies like Netflix in the Market, platforms must be able to analyze user behaviour and use it to adjust their platform accordingly.



To address the problem:

1. Provide personalized **recommendations**
2. Allow admin to access and edit their **watch history**
3. Show what users are often **searching**
    a. Can filter by region for cultural info
4. See users' **review** and rating tendencies

Dataset Description

Data Includes:

- 10,000+ users with subscription plans and activity status
- 1000+ movies with ratings and content types
- 100,000+ watch history records with duration and progress tracking
- 50,000+ recommendation logs with scoring and click tracking
- 25,000 Search logs with query patterns and location data
- 15,000 user reviews with ratings and verification status

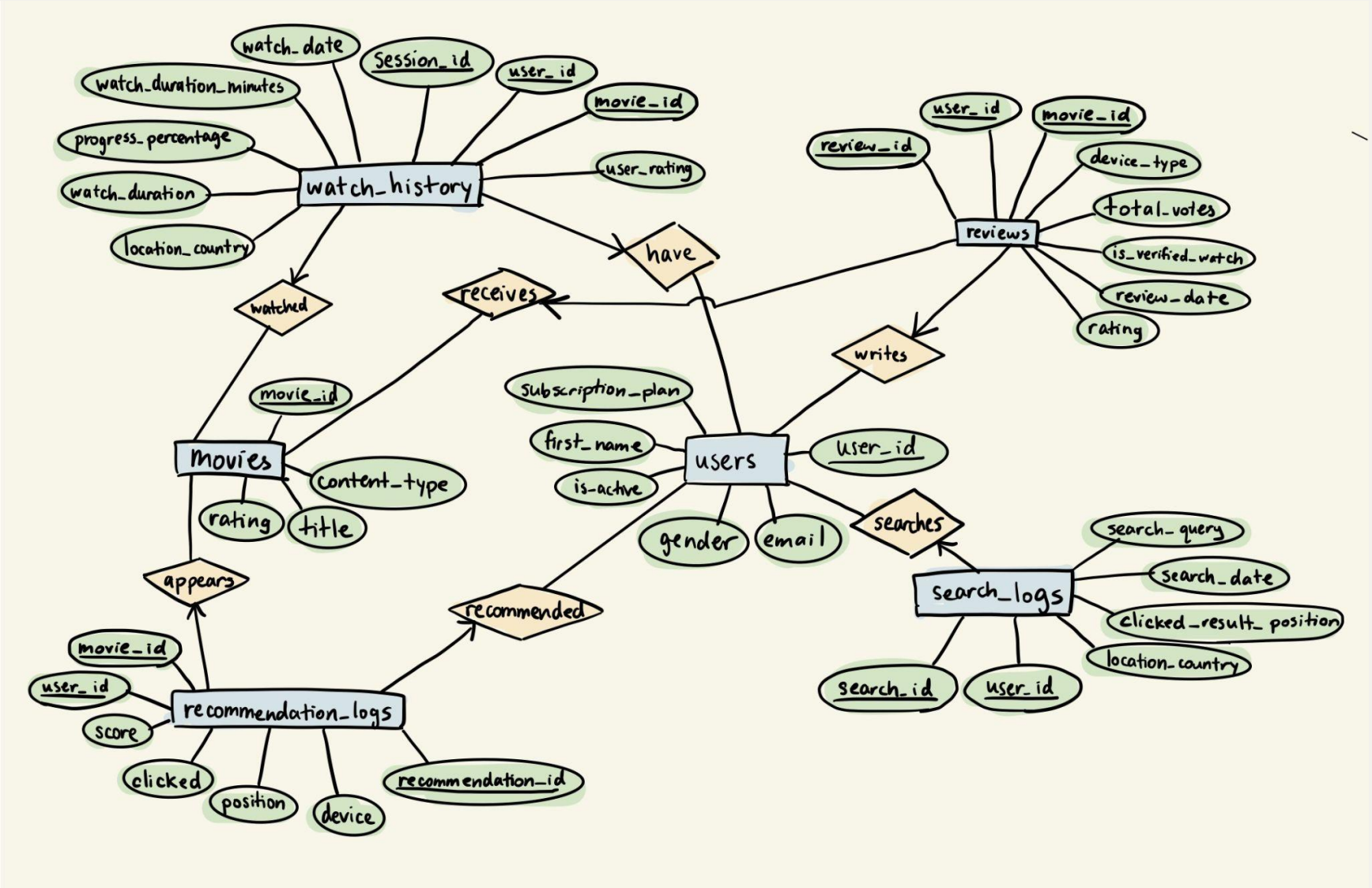
Source: Synthetic data generated to simulate real Netflix-like behavior patterns

Size: 200,000+ total records across 6 main tables

Why did they use synthetic data?

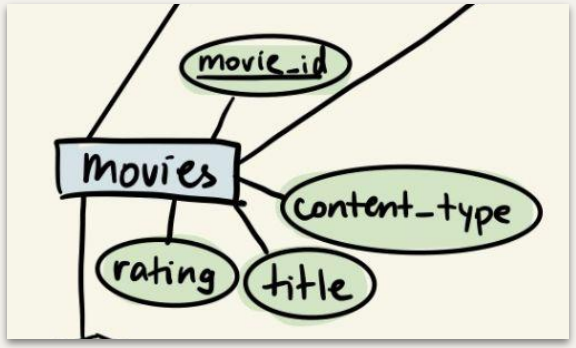
Real Netflix data is proprietary. We generated realistic data to demonstrate database capabilities and query performance at scale.

ER Diagram



- Note that there are no weak entities nor total participation (bold lines)
- there are single value constraints for all non-main tables
- makes sense for transactional systems (simply tracking events)

ER-to-Relational Mapping

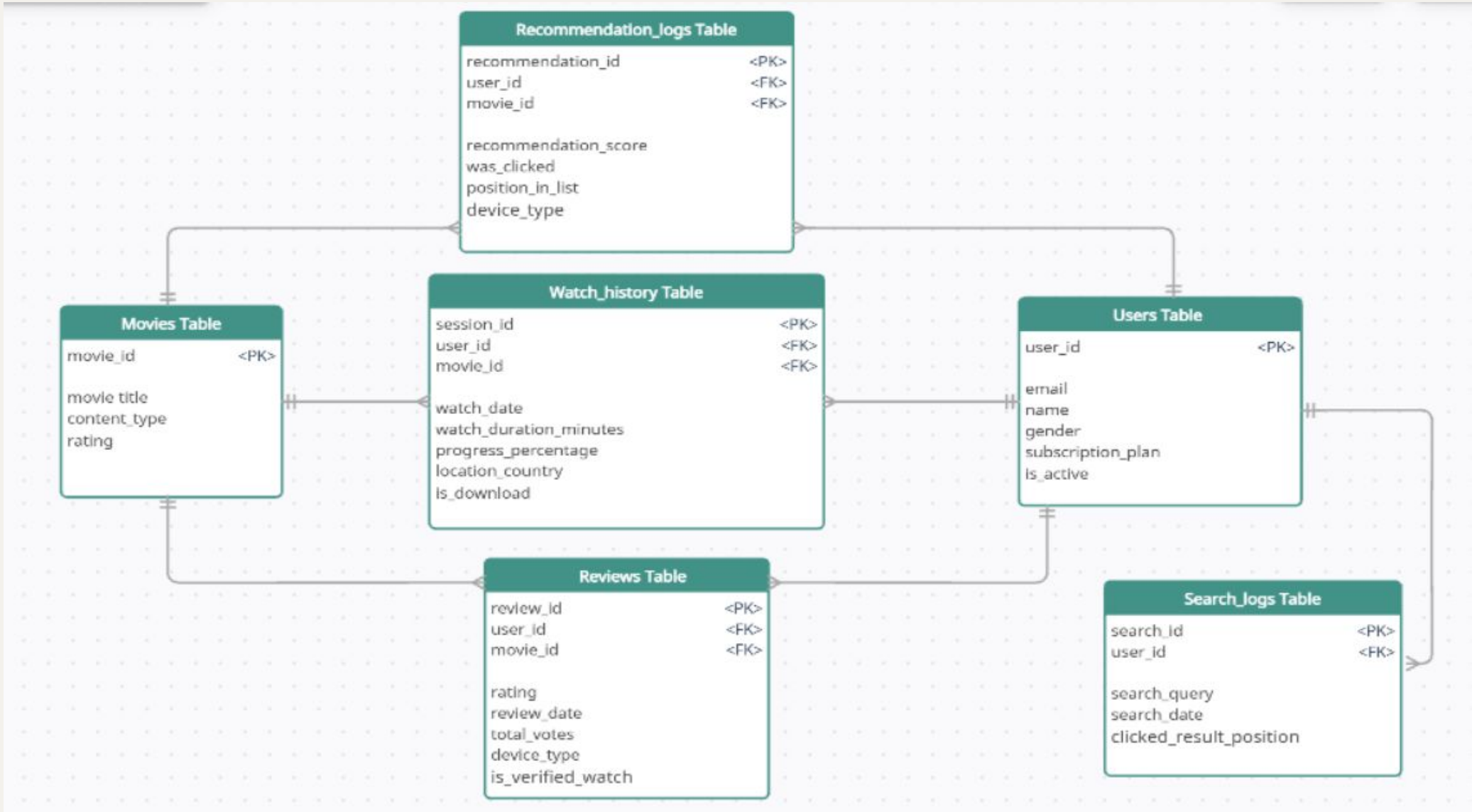


movie_id	title	content_type	rating
0001	Dragon Legend	Stand-up Comedy	TV-Y
0002	Storm Warrior	Stand-up Comedy	PG
0003	Fire Family	Movie	TV-MA
0004	Our Princess	Documentary	NC-17
0005	Warrior Mission	Documentary	TV-G
0006	Kingdom Phoenix	Movie	PG-13
0007	Battle Story	TV Series	TV-Y7
0008	Old House	Movie	TV-Y
0009	Dragon Empire	Movie	PG
0010	Dream War	Limited Series	TV-Y
0011	Day Dream	Movie	R
0012	Bright War	Movie	G
0013	Dream Journey	Movie	PG
0014	Little War	Movie	TV-14
0015	Hero Empire	Movie	TV-G

Each table in the database is modeled similarly to the above.

Since the ER diagram is quite simple, it also maps simply to the relational model.

Table Diagram

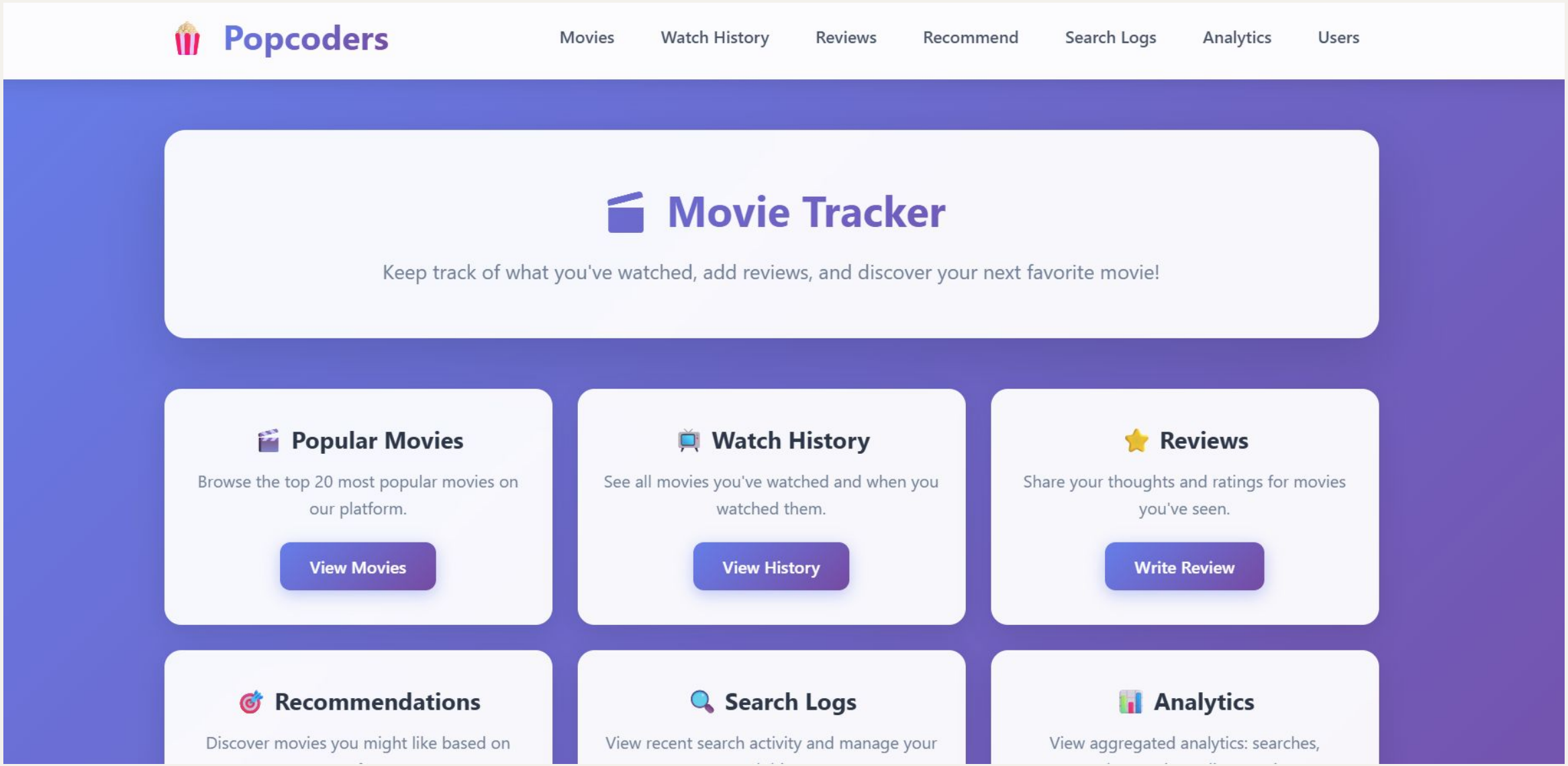


Normalization

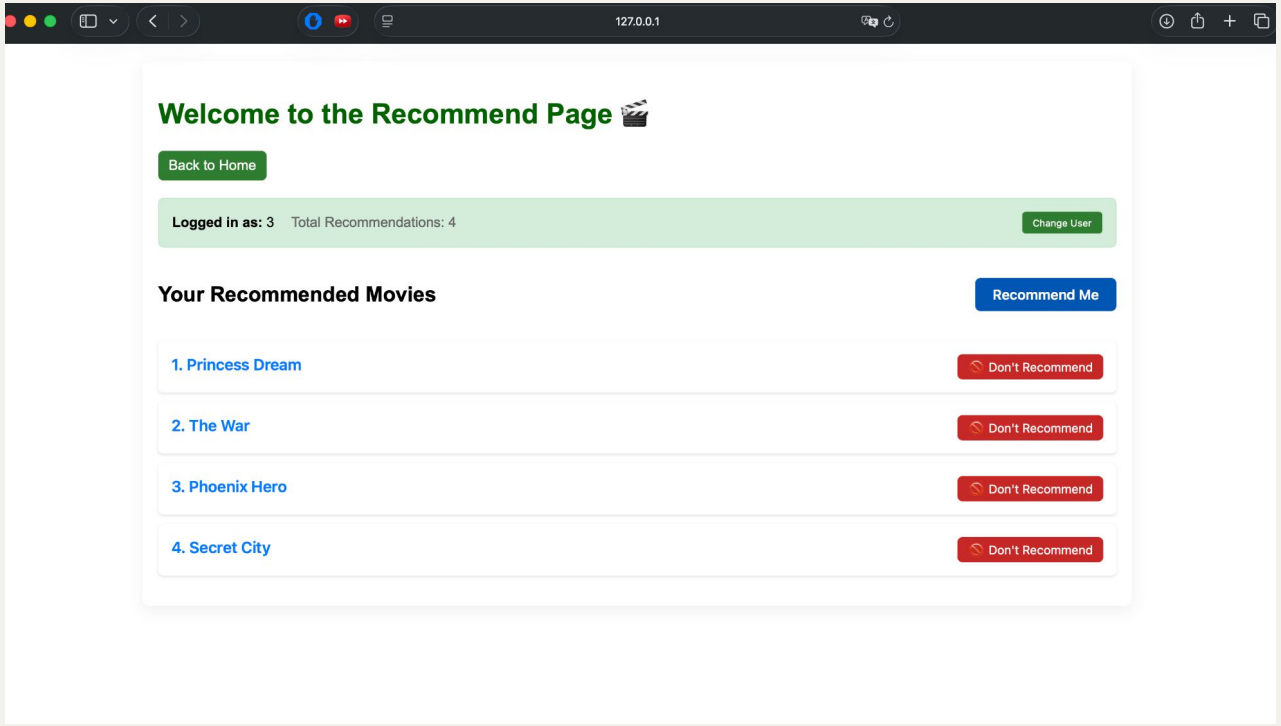
This design adheres to the **3NF**

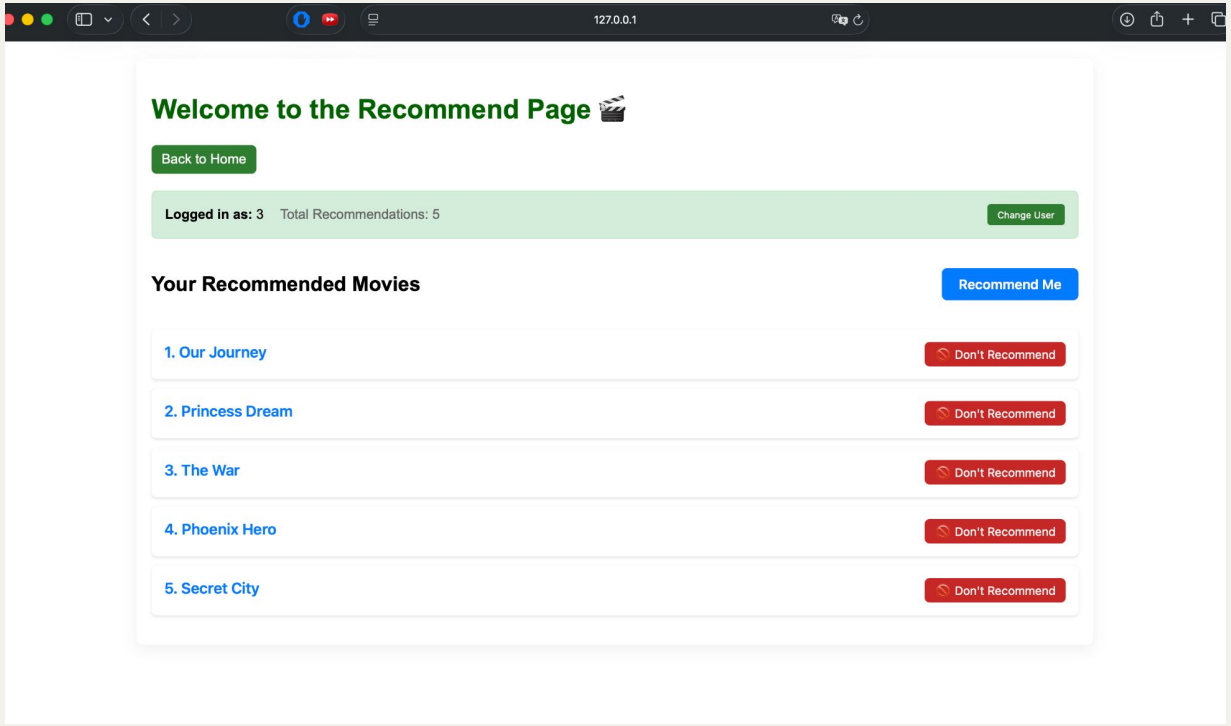
- It is **1NF** since it has atomic data types and a unique PK
- It is **2NF** since the PK is a single column, it is impossible to have a partial dependency. All other columns relate to this specific ID.
- It reached **3NF** by removing transitive dependencies; entity-specific data is assigned to users and movies tables, and they are linked strictly via FKs, which leaves only event-specific data in the logs

Features



Features





Interactive recommendation system where users receive personalized movie suggestions and can dynamically add new recommendations or remove unwanted ones.

Normalization

Before:

```
recommendation_id,user_id,movie_id,score,clicked,position,device
rec_000001,user_06326,movie_0771,,False,10,Tablet
rec_000002,user_02180,movie_0985,0.916,False,9,Mobile
rec_000003,user_03535,movie_0834,0.816,False,2,Tablet
rec_000004,user_05025,movie_0718,0.771,False,16,Mobile
```

After:

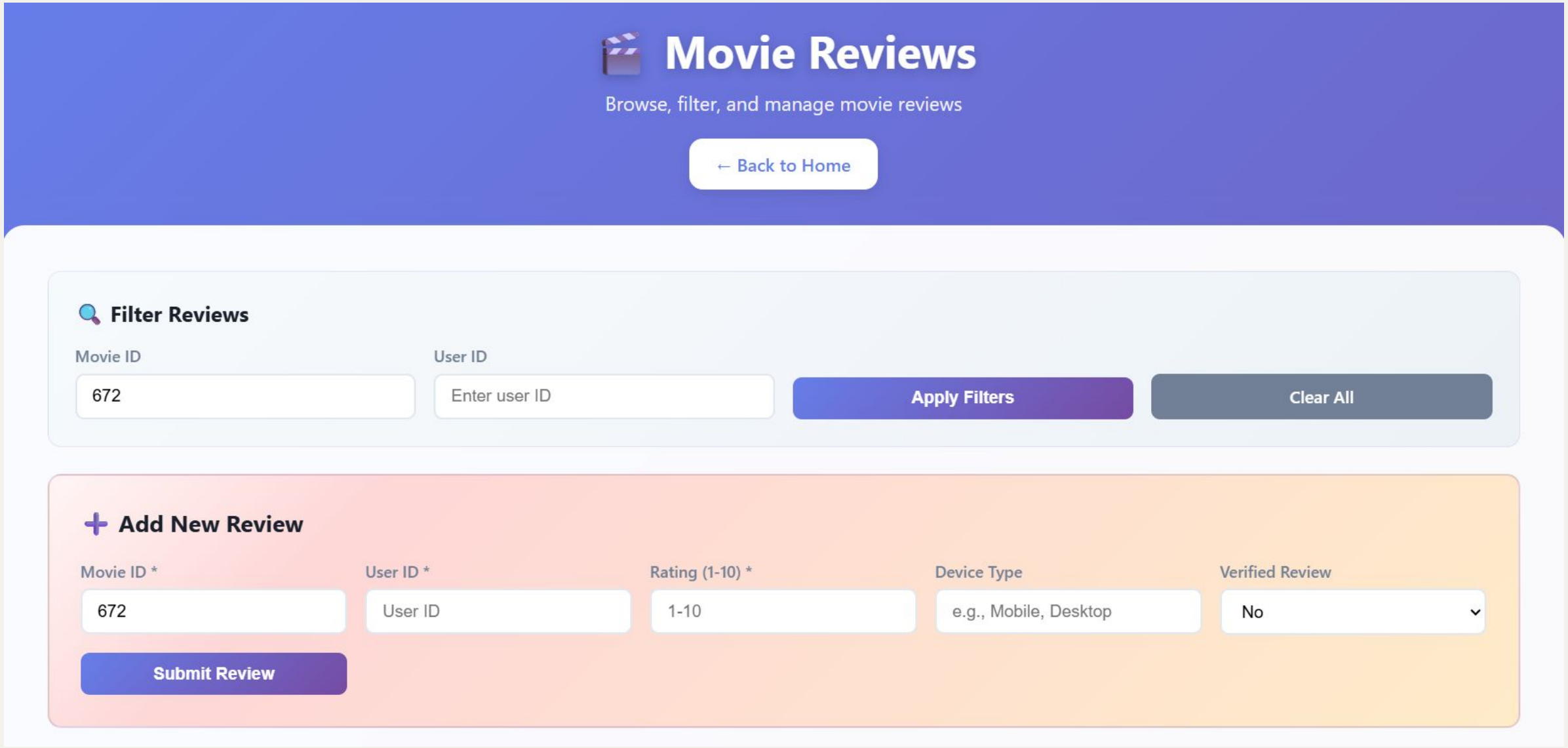
recommendation_ varchar(50)	* user int	* movie int	score decimal(5,3	* clicked tinyint(1)	position int	device varchar(50
rec_000002	2173	870	0.916	0	9	Mobile
rec_000003	3515	754	0.816	0	2	Tablet
rec_000004	4975	657	0.771	0	16	Mobile

Complex Queries

```
query = """SELECT rl.recommendation_id, rl.score, rl.position, rl.clicked,
    u.user_id, u.first_name,
    m.movie_id, m.title, m.rating
    FROM recommendation_logs rl
    JOIN users u ON rl.user_id = u.user_id
    JOIN movies m ON rl.movie_id = m.movie_id
    WHERE rl.user_id = %s
    ORDER BY rl.position
    LIMIT %s
"""
```

- Joins recommendation_logs, users, and movies tables
- Filters by user_id for Personalization
- Orders by position to show highest-priority recommendations
- Returns comprehensive data: user info, movie details, and recommendation scores

Reviews Feature



Normalization

Before:

```
review_id,user_id,movie_id,rating,review_date,device_type,is_verified_watch,total_votes
review_000001,user_07066,movie_0360,4,2025-03-29,Mobile,False,5.0
review_000002,user_02953,movie_0095,5,2024-07-19,Mobile,True,2.0
review_000003,user_05528,movie_0518,4,2025-02-11,Tablet,True,5.0
```

After:

review_id int	user_id int	movie_id int	rating tinyint	review_date date	device_type varchar(50)	is_verified_watch tinyint(1)	total_votes int
1	7066	360	4	2025-03-29	Mobile	0	5
2	2953	95	5	2024-07-19	Mobile	1	2
3	5528	113	4	2025-02-11	Tablet	1	5

Complex Queries

```
SELECT reviews.review_id, reviews.user_id, reviews.movie_id, movies.title AS movie_title,
       reviews.rating, reviews.review_date, reviews.device_type, reviews.is_verified_watch, reviews.total_votes
FROM reviews
LEFT JOIN movies ON reviews.movie_id = movies.movie_id
{where_sql}
ORDER BY reviews.review_id ASC
```

📋 All Reviews

#	USER ID	MOVIE ID	MOVIE TITLE	RATING	DATE	DEVICE	VERIFIED	VOTES	ACTIONS
1	8680	672	**Kingdom Dream**	⭐ 3	2024-06-18	Smart TV	✓ Yes	4	Edit Delete
2	67	672	**Kingdom Dream**	⭐ 3	2024-08-13	Tablet	✓ Yes	4	Edit Delete
3	5604	672	**Kingdom Dream**	⭐ 5	2024-02-08	Smart TV	✓ Yes	2	Edit Delete

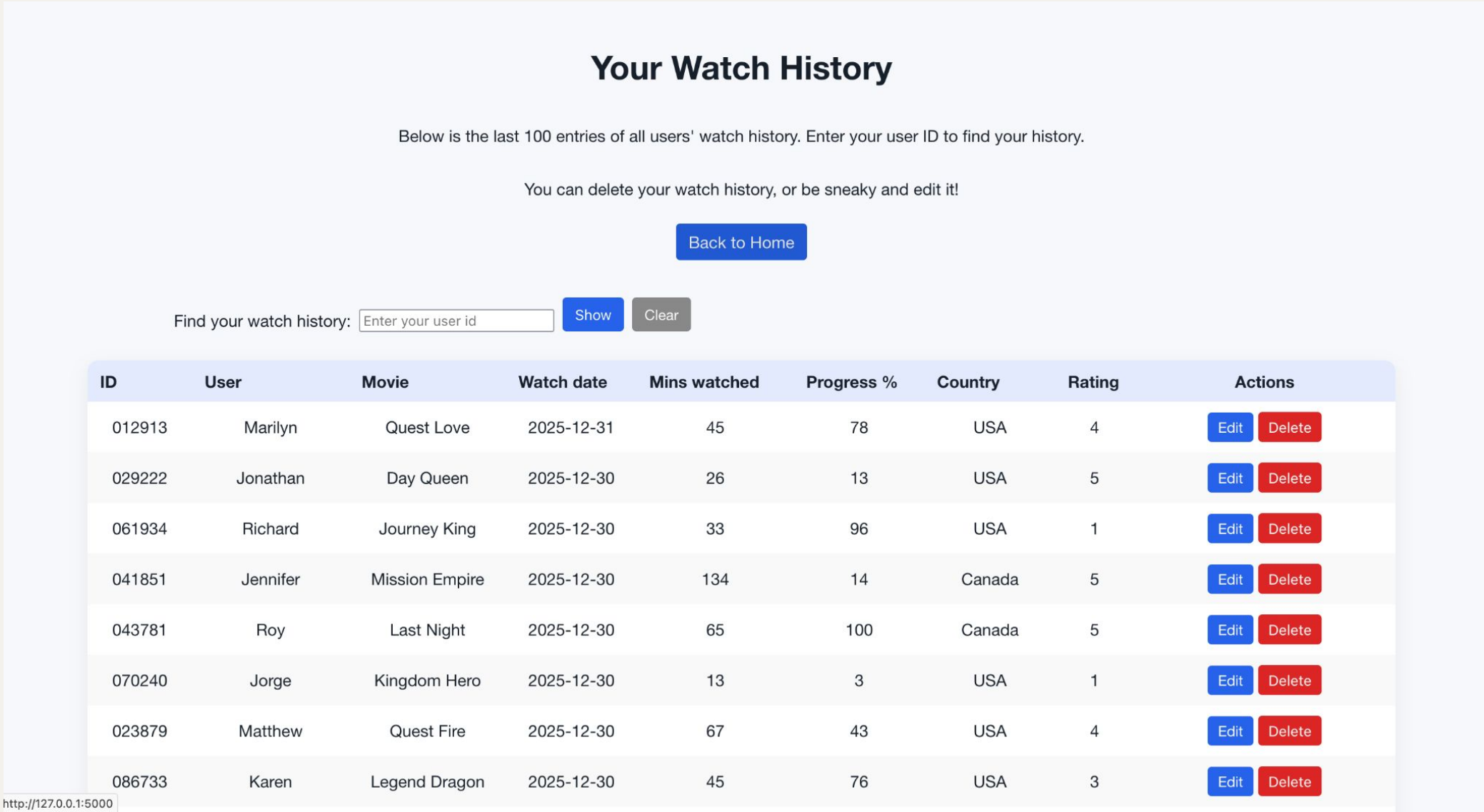
Complex Queries

```
SELECT reviews.review_id, reviews.user_id, reviews.movie_id, movies.title AS movie_title,
       reviews.rating, reviews.review_date, reviews.device_type, reviews.is_verified_watch, reviews.total_votes
FROM reviews
LEFT JOIN movies ON reviews.movie_id = movies.movie_id
{where_sql}
ORDER BY reviews.review_id ASC
```

- This query uses a LEFT JOIN to combine reviews with movie titles from two tables.
- It ensures all reviews are shown, supports filtering, and produces the data displayed in the application.
- We used LEFT JOIN because reviews must always be displayed, even if the related movie record is missing.

Watch History Feature

READ



Watch History Feature

UPDATE

Edit Watch History

User ID

09122

Movie ID

0766

Watch date

12 / 30 / 2025

Mins watched

134

Progress %

14 %

Rating

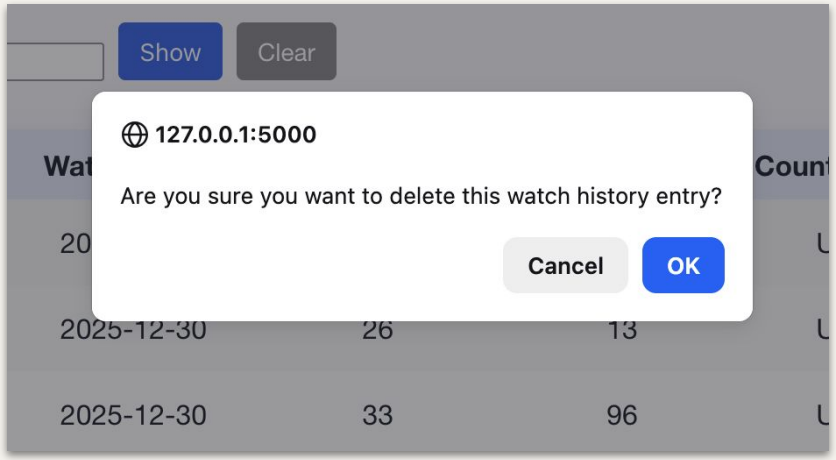
5

Country

Canada

Save Cancel

DELETE



Watch History Feature

Find your watch history: 00325 Show Clear

Finish These Movies

To the right are movies that you started, but didn't finish, while many other users did. Try watching it again, you might like it!

Big Warrior

Your progress	Average progress
67%	89%

Completed by 6 users

A Dream

Your progress	Average progress
0%	81%

Completed by 1 user

ID	User	Movie	Watch date	Mins watched	Progress %	Country	Rating	Actions
011322	Emily	Big Warrior	2025-03-01	64	67	Canada	4	Edit Delete
076698	Emily	A Dream	2024-04-19	29	0	Canada	4	Edit Delete

Complex Queries

- ➔ correlated subqueries in the SELECT clause
- ➔ subquery in the WHERE clause
- ➔ 3-table joins

Explanation coming up...

```
analysis_query = """
SELECT
    m.title AS Movie,
    wh.progress_percentage AS YourProgress,
    (
        SELECT ROUND(AVG(wh2.progress_percentage))
        FROM watch_history wh2
        WHERE wh2.movie_id = wh.movie_id
          AND wh2.user_id != wh.user_id
          AND wh2.progress_percentage >= 80
    ) AS OthersAvgProgress,
    (
        SELECT COUNT(DISTINCT wh2.user_id)
        FROM watch_history wh2
        WHERE wh2.movie_id = wh.movie_id
          AND wh2.user_id != wh.user_id
          AND wh2.progress_percentage >= 80
    ) AS OthersFinishedCount
FROM watch_history wh
JOIN movies m ON wh.movie_id = m.movie_id
JOIN users u ON wh.user_id = u.user_id
WHERE u.user_id = %s
  AND wh.progress_percentage < 70
  AND wh.movie_id IN (
      SELECT DISTINCT movie_id
      FROM watch_history wh2
      WHERE wh2.progress_percentage >= 80
        AND wh2.user_id != wh.user_id
  )
ORDER BY OthersFinishedCount DESC
LIMIT 3
"""
```

Complex Queries

creates a "new table" for each row in the main query and joins it based on row's movie. Then filters it to exclude user_id's entries, only show finished movies, then averages the progress for all entries.

combine wh, movies, and users tables

filter by inputted user id, and find movies that are incomplete (progress % below 70)

subquery to find distinct movies that others have completed (ie. not including inputted user)

check if any of the inputted user's incomplete movies are in the list of distinct movies others have completed

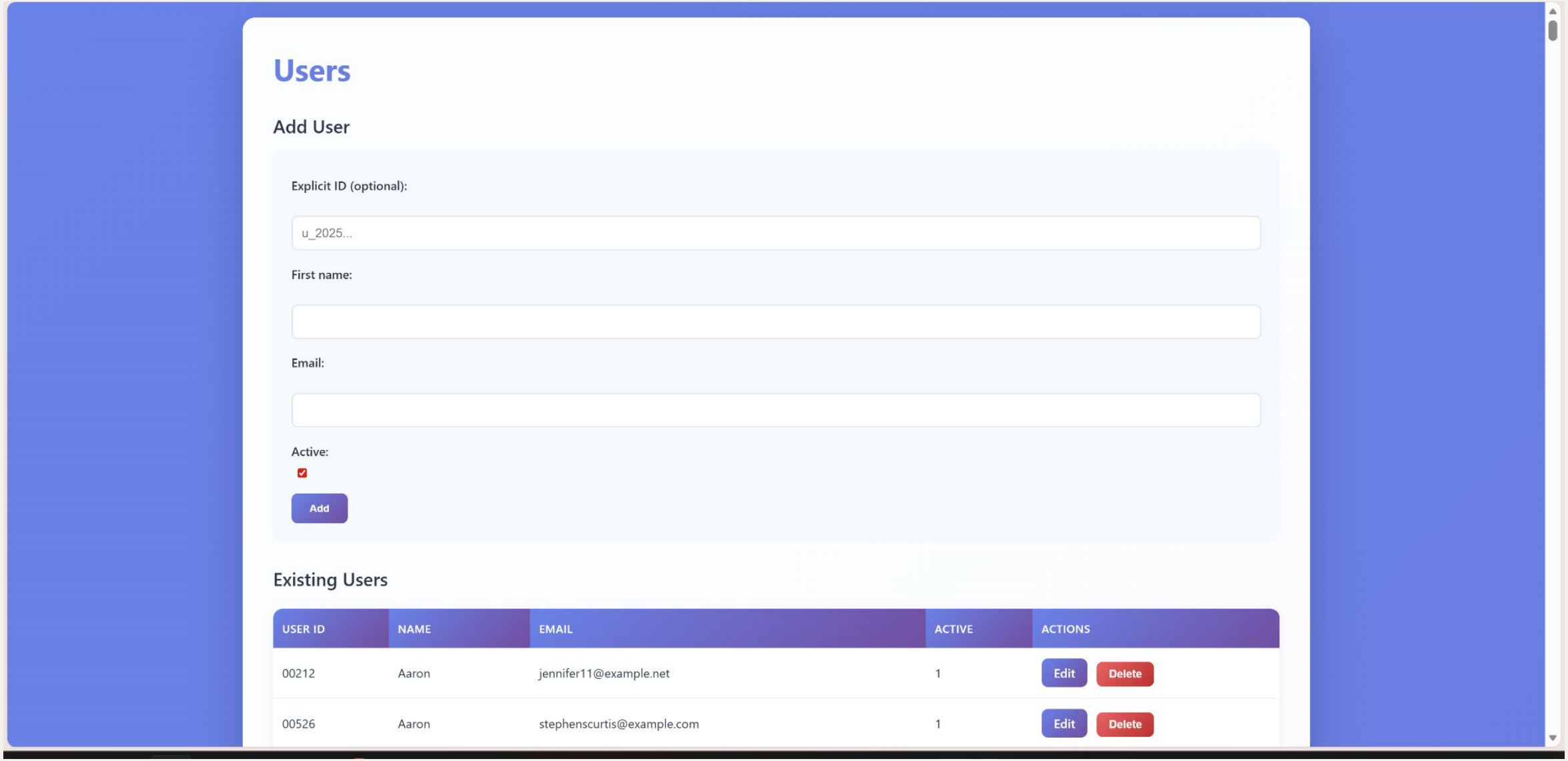
```
analysis_query = """
SELECT
    m.title AS Movie,
    wh.progress_percentage AS YourProgress,
    (
        SELECT ROUND(AVG(wh2.progress_percentage))
        FROM watch_history wh2
        WHERE wh2.movie_id = wh.movie_id
          AND wh2.user_id != wh.user_id
          AND wh2.progress_percentage >= 80
    ) AS OthersAvgProgress,
    (
        SELECT COUNT(DISTINCT wh2.user_id)
        FROM watch_history wh2
        WHERE wh2.movie_id = wh.movie_id
          AND wh2.user_id != wh.user_id
          AND wh2.progress_percentage >= 80
    ) AS OthersFinishedCount
FROM watch_history wh
JOIN movies m ON wh.movie_id = m.movie_id
JOIN users u ON wh.user_id = u.user_id
WHERE u.user_id = %s
  AND wh.progress_percentage < 70
  AND wh.movie_id IN (
      SELECT DISTINCT movie_id
      FROM watch_history wh2
      WHERE wh2.progress_percentage >= 80
        AND wh2.user_id != wh.user_id
  )
ORDER BY OthersFinishedCount DESC
LIMIT 3
"""
```

Complex Queries

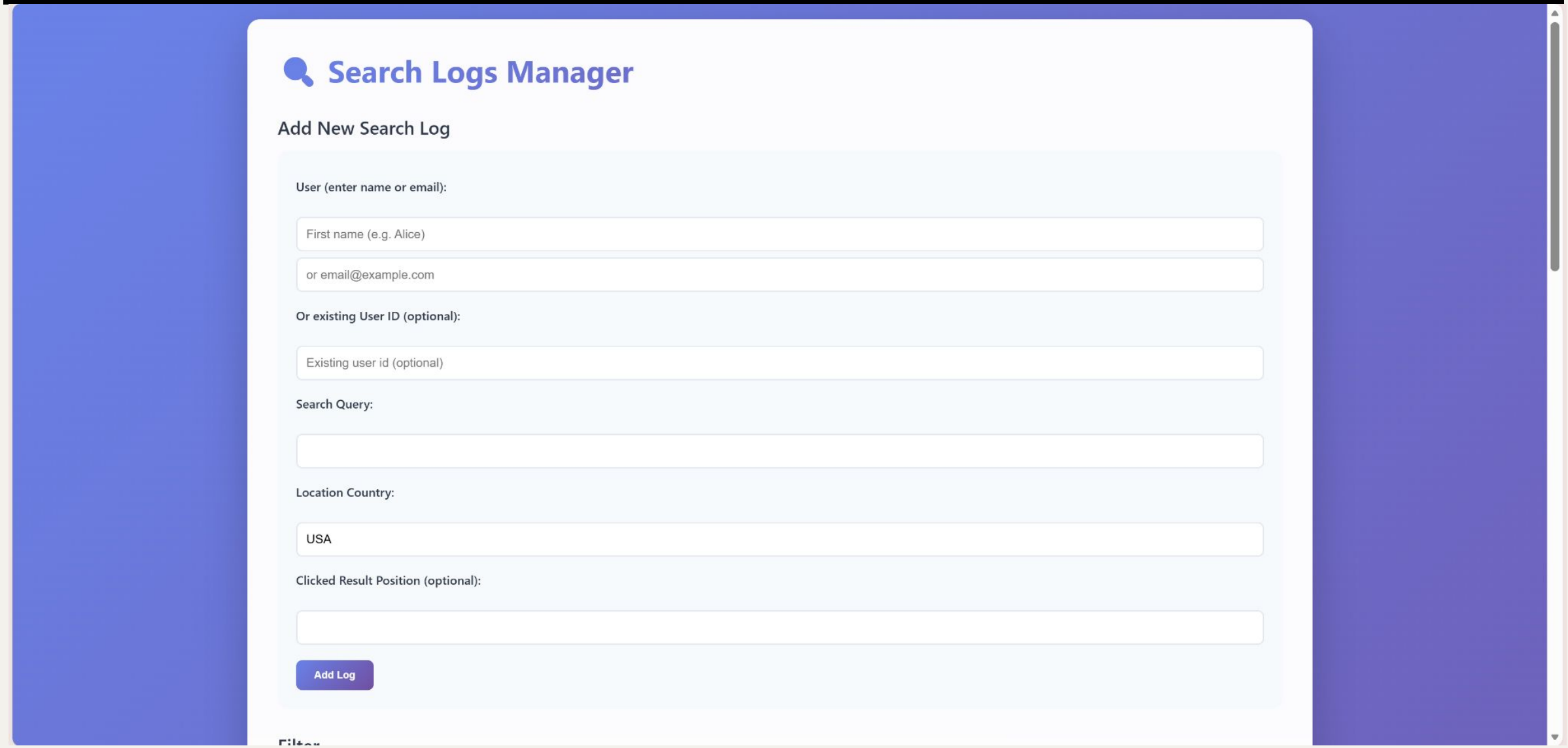
```
base_query = """
SELECT
    wh.session_id AS ID,
    COALESCE(u.first_name, u.user_id, wh.user_id) AS UserName,
    COALESCE(m.title, wh.movie_id) AS MovieTitle,
    wh.watch_date AS WatchDate,
    wh.watch_duration_minutes AS MinutesWatched,
    wh.progress_percentage AS ProgressPercentage,
    wh.location_country AS Country,
    wh.user_rating AS Rating
FROM watch_history wh
LEFT JOIN movies m ON m.movie_id = wh.movie_id
LEFT JOIN users u ON u.user_id = wh.user_id
"""
```

- ➔ outer join (left join)
- ➔ useful if a related movie was deleted, for example (even though cascading should handle this)

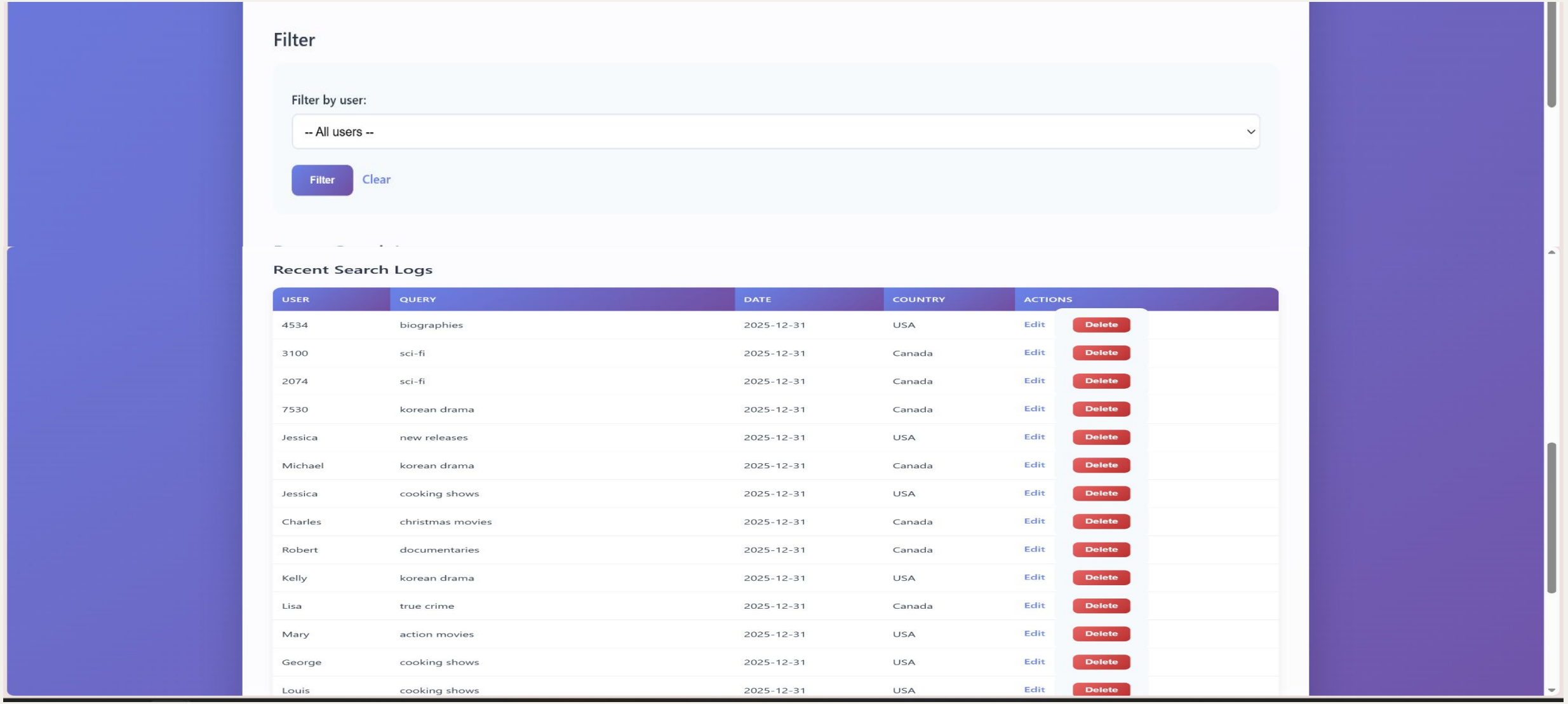
User Features



Search logs Features



Search logs Features



Analytics page Features

Analytics — User Summary

Demonstrates nested subqueries, GROUP BY, and joins across multiple tables.

USER ID	USER	TOTAL SEARCHES	TOTAL WATCH EVENTS	AVG RATING	TOP MOVIE
07393	07393	3	2	3.00	Hero Dragon
06288	06288	2	2	0.00	Warrior War
05593	05593	4	2	0.00	Phoenix Family
09556	09556	4	2	5.00	Ice Secret
04556	04556	4	2	0.00	Bright House
02328	02328	2	2	4.00	Secret Mission
01185	01185	2	2	0.00	War Storm
03171	03171	3	2	2.00	Empire Legend
00191	00191	2	1	0.00	Quest Quest
00198	00198	1	1	0.00	Our Storm
00203	00203	1	1	0.00	Mission Storm
00207	00207	5	1	0.00	Hero Journey
00263	00263	1	1	4.00	Old Phoenix
00302	00302	2	1	3.00	Fire War
00392	00392	4	1	1.00	Hero Ice

Country Summary (GROUP BY location)

Shows aggregated metrics per country using outer joins and a nested subquery to find top search query per country.

COUNTRY	TOTAL SEARCHES	TOTAL WATCH EVENTS	AVG RATING	TOP SEARCH QUERY	TOP MOVIE
USA	17495	255	0.00	sci-fi	Princess House
Canada	7509	109	0.00	stand up comedy	House Day

← Back to Home

Normalization

Before:

instead of storing user information like name and email in every search log entry, we separated it into a users table. This eliminates data redundancy :

search_logs:
- search_id (PK)
- user_id
- user_name ← Problem! Depends on user_id, not search_id ☹
- user_email ← Problem! Depends on user_id, not search_id
- search_query
- search_date
- location_country

Normalization

After:

search_id int	user_id varchar(50)	search_query text	search_date date	clicked_result_positio int	location_country varchar(100)
1	09864	classic movies	2024-03-22	2	Canada
2	u_test_1	sample search	2025-12-20	1	USA
3	u_test_1	sample search	2025-12-20	1	USA
4	01083	comedy shows	2024-12-14	4	USA

Complex Queries

```
# Join with users to show user names when available
base_query = """
    SELECT s.*, u.first_name AS user_name, u.email AS user_email
    FROM search_logs s
    LEFT JOIN users u ON s.user_id = u.user_id
"""
```

- Joins search_logs and users tables
- LEFT JOIN keeps all searches
- Displays user name and email for each search

```
if user_filter:
    count_sql = "SELECT COUNT(*) FROM search_logs s LEFT JOIN users u ON s.user_id = u.user_id WHERE (u.first_name LIKE %s OR u.email LIKE %s OR s.user_id = %s)"
    like_param = f"%{user_filter}%"
    count_cursor.execute(count_sql, (like_param, like_param, user_filter))
else:
    count_cursor.execute("SELECT COUNT(*) FROM search_logs")
```

- COUNT with conditional filtering
- Searches by name, email, or user_id
- Uses LIKE for pattern matching
- LEFT JOIN for user information

Complex Queries

```
# Query with actual watch and rating calculations - show users with watch history
sql = """
    SELECT
        w.user_id,
        w.user_id AS user_name,
        COUNT(DISTINCT s.search_id) AS total_searches,
        COUNT(DISTINCT w.session_id) AS total_watch_events,
        COALESCE(AVG(r.rating), 0.0) AS avg_rating,
        (SELECT m.title
         FROM watch_history w2
         JOIN movies m ON w2.movie_id = m.movie_id
         WHERE w2.user_id = w.user_id
         GROUP BY m.movie_id
         ORDER BY COUNT(*) DESC
         LIMIT 1) AS top_movie
    FROM watch_history w
    LEFT JOIN search_logs s ON w.user_id = s.user_id
    LEFT JOIN reviews r ON CONCAT('user_', w.user_id) = r.user_id
    GROUP BY w.user_id
    ORDER BY total_watch_events DESC
    LIMIT 50
"""
```

- Counts searches and watch events per user
- Calculates average rating with COALESCE
- Correlated subquery finds top movie for each user
- Three LEFT JOINs across watch_history, search_logs, reviews

Complex Queries

```
def analytics():
    Trailing whitespace
    # Country stats with nested subqueries - using derived table to avoid GROUP BY issues
    country_sql = """
        SELECT
            country,
            total_searches,
            (SELECT COUNT(*)
             FROM watch_history w
             WHERE COALESCE(w.location_country, 'Unknown') = country
            ) AS total_watch_events,
            0.0 AS avg_rating,
            (SELECT search_query
             FROM search_logs s2
             WHERE COALESCE(s2.location_country, 'Unknown') = country
             GROUP BY search_query
             ORDER BY COUNT(*) DESC
             LIMIT 1
            ) AS top_search_query,
            (SELECT m.title
             FROM watch_history w2
             JOIN movies m ON w2.movie_id = m.movie_id
             WHERE COALESCE(w2.location_country, 'Unknown') = country
             GROUP BY m.movie_id
             ORDER BY COUNT(*) DESC
             LIMIT 1
            ) AS top_movie
        FROM (
            SELECT
                COALESCE(location_country, 'Unknown') AS country,
                COUNT(*) AS total_searches
            FROM search_logs
            GROUP BY COALESCE(location_country, 'Unknown')
        ) AS country_searches
        ORDER BY total_searches DESC
        LIMIT 20
    """
```

- Derived table groups searches by country
- Three scalar subqueries for country metrics
- Multiple nested GROUP BY operations

Testing

- ➔ We performed only manual testing
- ➔ If **automated**, we could use frameworks like Selenium WebDriver
  - ◆ simulates real user interaction with web applications
  - ◆ can test our endpoints

Thank you!

Any questions?